Project 3 – Sprint 3 Planning Document

Sprint 3 Review Date: April 10th
Scrum Master for Sprint 3:

Sathkrith Segu

Section 1 - Sprint 2 Retrospective → Sprint 3 Operational Plan

Sprint 2 Structural Issue:

Our system for task breakdown was not precise or realistic enough, resulting in tasks that were still too large to be completed within a single class period. Although a rule was established to limit task size, it was not effectively applied because we consistently overestimated how much work could be completed during one period and failed to account for external factors such as workload from other classes and limited in-class time. As a result, tasks that were intended to be small and manageable often carried over multiple days, disrupting the expected daily progress and causing work to "snowplow". Additionally, because tasks were not consistently reevaluated or re-split once they exceeded the intended time limit, the board did not accurately reflect actual progress. This created a gap between planned work and real execution, leading to uneven productivity and a concentration of completed work toward the end of the sprint. What our team figured out was that our system lacked a mechanism to enforce realistic task sizing and adjustment based on actual working conditions, rather than initial estimates.

Sprint 2 Rule (exact wording):

All tasks must be small enough to be completed within one class period (approximately 45–60 minutes) and must include a clear, specific outcome. If a task is not completed within one class period, it must be immediately split into smaller tasks during the next Daily Scrum before any further work continues. Additionally, no task may remain in progress for more than one class period without being reassessed and resized.

What this will look like during daily work:

Each day, Sath will check the GitHub board to make sure that at least one task has been moved to done that day and that all tasks are appropriately sized for one class period. If tasks appear too large, he will flag them and the team will break the task down further before the next class day instead of waiting for the end of the sprint. Board review checkpoints will be used to verify that the backlog has been reviewed at least twice per week. During these checks, we will confirm that the tasks being worked on reflect the current sprint goals and are ordered by priority. If the backlog has not been updated Sath will make sure the team conducts a review before moving forward with other work.

Who is responsible for enforcement:

Sathkrith Segu

How will this be enforced? (select all that apply):

☒Daily Scrum checks

☒Board update checks

☒Task size enforcement

☐PR workflow enforcement

☐Other: Click or tap here to enter text.

When will enforcement occur during the Sprint?

Task review will occur at the end of each class period. During the final five-ten minutes of class, Sathkrith will make sure that tasks are on track to be completed by the end of the period. If not, then the tasks will be decomposed into smaller, more manageable tasks. Board review checkpoints will occur twice every week. During these review checkpoints, the backlog will be reviewed to ensure that the tasks being worked on meet our sprint goal.

Section 2 – Sprint Goal (MVP Completion + Stabilization)

Sprint Goal (By the end of Sprint 3, a user will be able to…):

By the end of Sprint 3, a logged in user will be able to explore and navigate an interactive map showing locations, be able to showcase a route, and be upload documents and pictures to a personal gallery that persists in the background even after a refresh.

What will NOT be included this Sprint (scope constraint):

This Sprint will not include route creation. Using a Google Maps API, we hope to outsource the work. Route creation is very time-consuming and would result in our team not being able to finish our MVP. We also do not plan on including energy statistics in this sprint and believe that postponing this feature would be helpful for our overall progress. Our team does not also plan on including a profile feature where a users image gallery is viewable to the public. For the sake of simplicity for this sprint, all images will be private to each user. Information and images will correlate to users stored in the Supabase.

Section 3 - Backlog Selection (2–4 items MAX)

Priority	Backlog Item	Reason it supports Sprint Goal	Why this is realistic this Sprint

L	**See Routes on Map**	Core to the Sprint Goal as it directly supports logged in navigation. Directly delivers user value by aligning with our product MVP and goal.	Requires API integration (Leaflet.js), a Supabase table for route/location data, a React map component, and connecting live data to map markers. Each step is its own task, being realistic once broken down into smaller steps.
M	**Gamify App (image uploads)**	Essential to the functional gallery aspect of the sprint goal, aligning with our further product goal of an engaging, game-like experience.	Users upload photos to a Supabase Storage bucket; URLs are stored in the database and displayed in a gallery grid. Depends on Sprint 2's auth (Supabase auth.uid() used for per-user filtering). Self-contained. These steps are realistic for the sprint as they are manageable increments.
Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.
Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.

Section 4 - Task Decomposition (All tasks ≤ 1 class period)

If you need less rows than provided just leave them blank. If you need more rows than provided consider if you can refactor tasks or if you have tasks that are outside of your current scope.

Task	Owner	Related Backlog Item	Done = (clear completion criteria)	Includes Testing ?

Research Leaflet.js vs Google Maps API vs Mapbox: compare cost, ease of React integration, and offline support. Write a 5-bullet decision doc in the GitHub Wiki.	**Aarush**	**See Routes on Map**	Done = decision recorded and shared with team.	**No**
Run 'npm install leaflet react-leaflet' in the project repo. Create a MapView.jsx component that renders a blank Leaflet map centered on a hardcoded latitude/longitude	**Yohaan**	**See Routes on Map**	Done = map renders in browser with no console errors.	**Yes**
Create a 'locations' table in Supabase with columns: id (uuid), name (text), latitude (float8), longitude (float8), description (text). Insert 4 seed rows manually using the Supabase table editor.	**Sriram**	**See Routes on Map**	Done = rows visible in Supabase dashboard.	**No**
Write a fetchLocations() async function using the Supabase JS client that runs 'supabase.from(locations).select(*)' and console.logs the result.	**Sriram**	**See Routes on Map**	Done = array of 4 location objects visible in browser DevTools console.	**Yes**
Import fetchLocations() into MapView.jsx. Use a useEffect hook to call it on mount and store results in useState. Pass each location's latitude/longitude to a Leaflet <Marker> component.	**Yohaan**	**See Routes on Map**	Done = 4 markers visible on map.	**Yes**
Add a Leaflet <Popup> inside each <Marker> that shows the location's name and description when clicked.	**Aarush**	**See Routes on Map**	Done = clicking any marker opens a popup with the correct text from Supabase.	**Yes**
Add MapView to the app's React Router so it's accessible at '/map'. Style the map container with Tailwind CSS so it fills the viewport height and matches the app's existing color scheme.	**Sathkrith**	**See Routes on Map**	Done = map page loads through the navigation link with no	**Yes**

			layout overflow.	
In Supabase Storage, create a bucket named 'gallery'. Set the bucket policy to allow authenticated users to insert and select their own files (using auth.uid() as the folder prefix).	**Yohaan**	**Gamify App**	Done = test upload works via Supabase dashboard.	**No**
Build an ImageUpload.jsx component with an <input type='file' accept='image/*'>, a preview <img> tag that shows a thumbnail before upload using URL.createObjectURL(), and an Upload button.	**Sathkrith**	**Gamify App**	Done = selecting a file shows a preview.	**Yes**
Write an uploadImage() async function that calls 'supabase.storage.from(gallery).upload()' with the file, then calls 'supabase.storage.from(gallery).getPublicUrl()' to get the URL, then inserts {user_id, image_url, created_at} into an 'images' table.	**Sriram**	**Gamify App**	Done = row appears in Supabase images table after upload.	**Yes**
Build a Gallery.jsx component that queries 'supabase.from(images).select(*).eq(user_id, user.id)' and maps results into a responsive grid of elements.	**Aarush**	**Gamify App**	Done = uploaded images appear in grid for the logged-in user.	**Yes**
Add a useEffect dependency on a refresh trigger so Gallery re-fetches after a new image is uploaded without requiring a full page reload.	**Sriram**	**Gamify App**	Done = uploading an image immediately appears in the gallery without refreshing.	**Yes**
End-to-end test: log in as two different test users, upload images as each, and confirm each user only sees their own photos.	**Sathkrith**	**Gamify App**	Done = no cross-user data leakage confirmed in browser.	**Yes**
Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to	Click or tap here to enter text.	Choose an item.

		enter text.		
Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Click or tap here to enter text.	Choose an item.

Section 5 - Testing Plan (Must occur throughout Sprint)

Which tasks include testing and how:

The tasks that include testing are:
Task #2 (MapView Component): map renders in browser without console errors
Task #4 (fetchLocations() function): make sure the array of four objects is visible in dev tools
Task #5 (Import FetchLocations into MapView): There are four markers visible on the map (corresponding to the table from the previous task)
Task #6 (Add a Leaflet Popup): Make sure clicking on any marker will open up a popup with the correct information from Supabase
Task #7 (Add MapView to React Router): The map page loads through the navigation bar with no overflow.
Task #9 (Build ImageUpload Component): Selecting a file shows a corresponding preview
Task #10: (uploadImages() function): After uploading an image, a row should show up in the Supabase table (visible through the Supabase interface)
Task# 11 (Build gallery.jsx component): Uploaded images appear in the grid for the logged in user
Task #12 (add useEffect dependency): Uploading an image will immediately appear in the gallery without refreshing
Task #13 (Test): described in task above

Who is responsible for testing:

Sriram Marsanapalle Kalle

How testing ensures Definition of Done is met before marking Done:

Testing will be conducted in periods all throughout the Sprint rather than just have a designated testing period at the end. By ensuring that each version of a task is done before marking it complete, a testing period allows ample time to “break” the code giving confidence to everyone on the team. Furthermore, as our group is planning on partner coding, by giving

our code to the other pair on the team we allow everyone to become familiar with what content is functioning.

Click or tap here to enter text.

Click or tap here to enter text.

Section 6 - Workflow & Board Discipline

When will the board be updated each day:

The board will be updated at the end of each period: in the final five to ten minutes of class, Sathkrith will conduct the Task Reviews where he will make sure that tasks are on track to be completed by the end of the period). If Sath discovers that tasks are not on track to be completed by the end of the period, then the tasks will be decomposed into smaller, more manageable tasks (board will be updated).

How tasks will move (To Do → In Progress → Done):

Tasks will remain in the "To Do" Section until at least one commit has been made towards the completion of the task. At this stage, the task will move to the "In Progress" Section. When the task meets our team's Definition of Done and has been tested thoroughly, the task will move to the "Done" Section.

Rule to prevent last-minute work completion:

Before our team's Daily Scrum Meetings, the Scrum Master (Sathkrith Segu) will have a personal conversation with each developer (1-2 minutes long) on how the developer's work is going. During this meeting, the developer can raise concerns about any issues that have risen in the last 24 hours, and the Scrum Master can offer personalized support. If there are any issues, the team can focus on solving them during the Daily Scrum Meeting. This increased transparency will aid our team resolve issues early instead of having the issues pile up throughout the Sprint.

Section 7 - Dependencies & Parallel Work Plan

Identify one key dependency:

Our Sprint focuses heavily on map creation, as such, API integration either through Leaflet (Leaflet React) or Google Maps API is the highest dependency.

What work can happen in parallel:

Work that can happen in parallel would be image uploading, backend creation for storing information relating to routes and creation as well as page creation and display option for the map itself (even if it's not input yet).

What work must wait and why:

Work that must wait depends on the map integration. As such this work would relate to being able to display routes based on user input as well as a map that the user can freely navigate. This is because both of these tasks are visual and must depend on the testers actually having a map (which requires working Leaflet) that we can test with.

Section 8 - Risk & Adjustment Plan

Primary risk to completing Sprint Goal:

The biggest risk to this Sprint is time estimation errors again (our documented root cause from Sprint 2). Specifically, integrating an external map library (Leaflet + react-leaflet) is the hardest task for our team as we haven't used it before. Wrong tile provider configuration or React-Leaflet version mismatches could easily take up more than a class period, just on debugging, leaving no time for the Supabase integration tasks or some of our other "smaller tasks" such as image uploading and display.

Adjustment trigger (If _____ happens by _____, we will _____):

If the leaflet setup process takes longer than one class period, Sathkrith (our Scrum Maser) will flag It in our next scrum meeting so that the team can immediately split the content, according to our Sprint plan.

Section 9 – Team Commitment

☒We confirm this plan is realistic and aligned to our Sprint Goal.

☒All tasks meet the ≤1 class period requirement.

☒Testing is integrated into our workflow (not delayed).